



1. Локальный Git и Git-хостинг — это разные уровни

До этого мы в основном работали с Git внутри локального проекта: смотрели `git diff`, проверяли состояние файлов и использовали историю изменений. Теперь появляется следующий уровень — Git-хостинг. Это уже серверная платформа, на которой хранится удалённый репозиторий и организуется совместная работа команды.

Когда проект находится на GitHub, GitLab или другой подобной платформе, агенту уже недостаточно понимать только локальное состояние файлов. Ему может понадобиться узнать, какая ветка опубликована на сервере, существует ли Pull Request, кто его открыл, какие проверки запущены и можно ли объединять изменения.

Поэтому интеграция с Git-хостингом расширяет рабочую область агента. Локальный Git сообщает ему, что происходит в рабочей копии, а Git-хостинг показывает состояние командного процесса вокруг этих изменений.

2. Удалённый репозиторий как источник состояния проекта

Удалённый репозиторий я рассматриваю не просто как место хранения резервной копии кода. В современной разработке он становится центральной точкой взаимодействия команды.

Именно там находятся опубликованные ветки, Pull Request, обсуждения изменений, автоматические проверки и история объединения кода. Поэтому для агента Git-хостинг является ещё одним внешним источником текущего состояния проекта.

Например, локально агент может увидеть, что тестируемая функция была изменена. Но только Git-хостинг может сообщить ему, что для этой ветки уже существует открытый Pull Request и что одна из серверных проверок завершилась ошибкой.

Получается, что при интеграции с Git-хостингом агент начинает работать не только с файлами, но и с процессом разработки вокруг этих файлов.

3. Ветка как изолированная область изменения

При агентной разработке особенно важно не позволять системе изменять основную ветку проекта напрямую без необходимости. Рабочая ветка создаёт отдельную область, где изменения можно подготовить и проверить до объединения с основной версией.

Для агента это удобно ещё и потому, что появляется явная граница задачи. Можно сказать, что все изменения текущей работы должны находиться в конкретной ветке, а состояние основной ветки используется как точка сравнения.

Когда агент получает задачу вроде «обнови зависимость и подготовь изменение», логическим результатом его работы должна быть не просто изменённая рабочая директория. Мы хотим получить контролируемый набор изменений, который можно передать дальше в процесс проверки.

И именно здесь начинает появляться Pull Request.

4. Pull Request как объект рабочего процесса

Pull Request, или PR, — это не просто кнопка «добавить код в main». Я рассматриваю его как отдельный объект процесса разработки.

Он связывает исходную и целевую ветки, содержит описание изменений, diff, комментарии, результаты review и статусы автоматических проверок. Через PR команда принимает решение о том, готово ли изменение к объединению.

Для агента это особенно удобно, потому что задача становится формализованной. Он может подготовить изменения, создать PR и затем работать уже не с абстрактной фразой «мой код готов», а с конкретным объектом на Git-хостинге.

Hermes может взаимодействовать с GitHub через существующие GitHub-возможности и CLI. В актуальной документации Hermes отдельно отмечается, что для GitHub предпочтительна интеграция через встроенные `github/*` skills и GitHub CLI `gh`; GitHub даже намеренно не включён в обычный MCP-каталог Hermes, потому что связка с `gh` предоставляет более развитый рабочий процесс.

5. Что агент может делать с PR

Когда агент подключён к Git-хостингу, работа с PR может включать уже несколько уровней взаимодействия.

Он способен получить список открытых Pull Request, определить ветки, прочитать описание, получить diff и посмотреть результаты проверок. При достаточных разрешениях он может также создать новый PR или оставить комментарий.

Но техническая возможность действия ещё не означает, что агент обязательно должен её использовать. Как и с другими инструментами, полезно разделять операции чтения и изменения.

Например, просмотр состояния PR и CI является относительно безопасной read-only операцией. Создание PR уже изменяет состояние Git-хостинга. Слияние PR затрагивает основную историю проекта ещё сильнее.

Поэтому уровень полномочий агента должен соответствовать его роли.

6. CI/CD-триггер как реакция на событие

Теперь свяжем Git-хостинг с CI/CD.

CI/CD-процесс обычно не запускается сам по себе случайным образом. Его инициирует определённое событие, которое называется trigger.

Например, автоматическая проверка может запускаться после публикации нового commit в ветку или после создания и обновления Pull Request. В этот момент Git-хостинг передаёт информацию системе автоматизации, а та запускает определённый workflow.

Для нас важно понимать саму причинно-следственную связь. Агент публикует изменение в удалённой ветке. На Git-хостинге происходит событие. Это событие запускает CI. CI выполняет тесты, линтеры, сборку или другие проверки. После этого результат снова становится доступным через Git-хостинг.

Получается замкнутый рабочий цикл, в котором Git-хостинг выполняет роль координационного центра между кодом и автоматизацией.

7. Статус CI как обратная связь для агента

После создания PR агент не должен считать задачу завершённой только потому, что код успешно отправлен на сервер.

На стороне CI могут запускаться проверки, которые невозможно было полностью воспроизвести локально. Например, проект может тестироваться в другой операционной системе, в контейнере, на другой версии Python или с дополнительными интеграционными сервисами.

Поэтому статус CI становится новым источником обратной связи.

Если проверки завершились успешно, агент получает подтверждение, что изменение прошло заданные серверные условия. Если проверка упала, необходимо различать сам факт failure и его причину.

В Hermes это можно использовать как продолжение уже знакомого агентного цикла: получить состояние PR, посмотреть неуспешную проверку, прочитать её результат и только после этого решать, какое действие требуется дальше.

8. Pull Request не равен успешному CI

Здесь важно не смешивать два разных состояния.

PR может быть создан успешно, но его проверки ещё выполняются. Он может существовать, но содержать упавший CI. И наоборот, все автоматические проверки могут завершиться успешно, но PR всё ещё может ожидать человеческого review или выполнения других правил репозитория.

Поэтому агенту недостаточно проверить только наличие PR.

Необходимо понимать разные состояния процесса: изменение опубликовано, автоматическая проверка запущена, проверка завершена, review проведён, условия объединения выполнены.

В реальной системе Git-хостинг может иметь несколько параллельных workflow. В самом репозитории Hermes, например, GitHub Actions используется для отдельных тестов, typecheck и других CI-проверок.

То есть фраза «CI зелёный» обычно должна означать не один случайный job, а успешное состояние тех проверок, которые действительно обязательны для конкретного проекта.

9. Polling и webhook как два режима интеграции

Если агент должен узнавать об изменениях в Git-хостинге, существует два принципиально разных подхода.

При polling агент сам периодически обращается к серверу и спрашивает, появились ли новые Pull Request или изменился ли статус проверки. Такой подход проще и может работать даже тогда, когда Hermes находится внутри локальной сети без публичного адреса. В официальном руководстве Hermes именно polling через cron предлагается как один из способов построения агента для проверки GitHub PR.

Другой подход — webhook. Здесь направление меняется: Git-хостинг сам отправляет событие Hermes, когда PR был создан или обновлён. Hermes может сразу получить событие, извлечь diff, проанализировать его и отреагировать. В документации Hermes есть отдельный сценарий автоматического PR-review именно через GitHub webhook.

Polling удобен своей простотой, а webhook уменьшает задержку между событием и реакцией. Это уже архитектурный выбор интеграции, а не вопрос качества самой языковой модели.

10. Роль Hermes в Git-процессе

В нашей дальнейшей работе я хочу рассматривать Hermes как участника существующего Git-процесса, а не как систему, которая заменяет Git-хостинг.

Hermes может подготовить изменение локально, использовать Git, обратиться к GitHub через `gh`, создать или исследовать Pull Request и получить состояние CI. GitHub при этом продолжает отвечать за удалённый репозиторий, PR, проверки и правила объединения.

Современный Hermes может работать и с внешними системами через MCP, причём каждый MCP-сервер превращается в отдельный toolset с контролируемым набором доступных операций. Но конкретно для GitHub документация Hermes рекомендует встроенный workflow через GitHub CLI и соответствующие skills, а не подключение максимально широкого внешнего инструментария без необходимости.

Для нас главный принцип заключается в том, что агент не должен считать отправку кода конечной точкой задачи. В командной разработке изменение проходит через удалённую ветку, Pull Request и автоматические проверки. Только после получения этих результатов можно объективно говорить о состоянии подготовленного изменения.

В следующей практической работе мы именно эту цепочку и выполним: подготовим изменение через Hermes, создадим Pull Request на Git-хостинге и затем отдельно получим статус запущенного CI, не подменяя серверную проверку локальным предположением агента.
